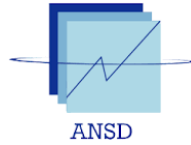


République du Sénégal

Ministère de l'Économie, du Plan et de la Coopération

★ ★ ★ ★ ★ ★ ★ ★ ★ ★



Agence nationale de la Statistique et de la Démographie



École nationale de la Statistique et de l'Analyse économique Pierre
NDIAYE

DEEP LEARNING

RECONNAISSANCE VOCALE AVEC DEEP LEARNING

Rédigé par :

Poko Ibrahim NOBA

Ndeye Salla TOURE

Aïssatou GUEYE

Moise ATTISO

Abdou BA

Élèves Analystes Statisticiens

Sous la direction de :

Mme Fatou SALL

Data Scientist

Mars 2026

Table des matières

Introduction générale	3
1 Architectures deep learning pour la reconnaissance vocale	5
1.1 Deep Speech 2 : l'approche end-to-end par RNN	5
1.1.1 Architecture	5
1.1.2 Entraînement et résultats	7
1.2 Wav2Vec 2.0 : l'apprentissage auto-supervisé	7
1.2.1 Architecture	7
1.2.2 Scoring pondéré dans T.Speak	9
1.2.3 Résultats et implications	9
1.3 Whisper : généralisation à l'échelle et au multilinguisme	9
1.3.1 Architecture	9
1.3.2 Paramètres d'entraînement et robustesse dans T.Speak	10
1.3.3 Résultats et robustesse zéro-shot	11
1.4 Comparaison des trois architectures dans le contexte de T.Speak	11
2 Architectures deep learning pour la reconnaissance vocale	12
2.1 Deep Speech 2 : l'approche end-to-end par RNN	12
2.1.1 Architecture	12
2.1.2 Entraînement et résultats	14
2.2 Wav2Vec 2.0 : l'apprentissage auto-supervisé	14
2.2.1 Architecture	14
2.2.2 Scoring pondéré dans T.Speak	16
2.2.3 Résultats et implications	16
2.3 Whisper : généralisation à l'échelle et au multilinguisme	16
2.3.1 Architecture	16
2.3.2 Paramètres d'entraînement et robustesse dans T.Speak	17
2.3.3 Résultats et robustesse zéro-shot	18
2.4 Comparaison des trois architectures dans le contexte de T.Speak	18
3 Fine-tuning et adaptation au contexte africain francophone	19
3.1 Pourquoi le fine-tuning est nécessaire	19

3.1.1	Interférences phonétiques identifiées dans T.Speak	19
3.1.2	Spécificités grammaticales et lexicales	20
3.2	Jeux de données disponibles pour le contexte africain	21
3.2.1	Common Voice (Mozilla)	21
3.2.2	Mozilla Common Voice — Langues africaines	21
3.2.3	Corpus T.Speak (construction en cours)	21
3.3	Protocoles de fine-tuning	22
3.3.1	Fine-tuning de Whisper sur accents africains	22
3.3.2	Fine-tuning de Wav2Vec 2.0 sur phonèmes africains	23
3.4	Métriques d'évaluation dans T.Speak	24
3.4.1	Word Error Rate (WER)	24
3.4.2	Score de prononciation pondéré	25
3.4.3	Métriques NLP complémentaires	25
3.4.4	Latence de traitement	25

Introduction générale

La capacité à comprendre et à évaluer la parole humaine constitue l'un des défis les plus ambitieux de l'informatique moderne. La reconnaissance automatique de la parole (RAP), connue en anglais sous le sigle *ASR* pour *Automatic Speech Recognition*, désigne l'ensemble des techniques permettant à une machine de transcrire un signal audio en texte. Depuis les premières tentatives dans les années 1950 jusqu'aux systèmes commerciaux actuels, ce domaine a connu des transformations profondes, dont la plus récente et la plus décisive est l'introduction du deep learning.

Pendant plusieurs décennies, les systèmes de reconnaissance vocale reposaient sur une combinaison de modèles de Markov cachés (HMM) et de mélanges gaussiens (GMM), complétée par des traitements manuels du signal tels que les coefficients cepstraux en fréquence de Mel (MFCC). Ces architectures, performantes dans des conditions contrôlées, se heurtaient à des limites structurelles face à la variabilité naturelle de la parole, aux environnements bruités et à la diversité linguistique. Le taux d'erreur restait élevé dès que les conditions d'enregistrement s'éloignaient du cadre d'entraînement (??).

Le tournant s'est produit au début des années 2010, lorsque les réseaux de neurones profonds ont démontré leur supériorité sur les approches classiques en modélisation acoustique (?). Cette rupture a ouvert la voie à une série d'innovations architecturales majeures : réseaux convolutifs (CNN), réseaux récurrents à longue mémoire (LSTM), puis modèles *end-to-end* capables d'apprendre directement depuis la forme d'onde brute jusqu'au texte, sans pipeline intermédiaire (??). Plus récemment, les architectures Transformer ont encore déplacé les frontières du possible, notamment grâce à leur capacité à exploiter de vastes corpus non annotés via l'apprentissage auto-supervisé (??).

Aujourd'hui, des systèmes tels que Wav2Vec 2.0 ou Whisper atteignent des niveaux de performance comparables, voire supérieurs, à ceux d'un transcrivoteur humain sur certaines tâches de référence. Pour autant, des défis substantiels demeurent : la généralisation aux langues peu dotées en données, la robustesse aux conditions acoustiques dégradées, le coût computationnel élevé et la difficulté d'interprétation des modèles, souvent perçus comme des « boîtes noires » (?).

C'est précisément dans ce contexte que s'inscrit le projet **T.Speak**, application mobile d'ap-

prentissage de l'anglais oral conçue pour les locuteurs francophones d'Afrique de l'Ouest. T.Speak illustre de manière concrète la mise en œuvre des architectures théoriques étudiées dans ce rapport : le module `WhisperTranscriber` (`transcriber.py`) intègre `faster-whisper` avec support du fine-tuning sur accents africains et calcul du taux d'erreur de mots (WER) via `compute_wer()` ; le module `Wav2VecScorer` (`scorer.py`) exploite `facebook/wav2vec2-base-960h` pour évaluer la qualité acoustique phonème par phonème, avec des conseils ciblés sur les phonèmes difficiles pour les locuteurs francophones (sons TH, R rétroflexe, V vs B) ; le module NLP (`nlp_analyzer.py`) évalue la grammaire via `spaCy` et le vocabulaire via la grille CEFR (de A1 à C2) ; enfin, le `ConversationGenerator` (`generator.py`) orchestre un grand modèle de langage pour produire un retour personnalisé, contextualisé aux réalités socio-économiques africaines (Dakar, Lagos, Abidjan) et aux scénarios professionnels locaux (pitch de startup, entretien d'embauche).

Ce pipeline complet — du signal audio brut au feedback textuel en moins de trois secondes — constitue une application directe des travaux académiques fondateurs que ce rapport a pour vocation de synthétiser.

Problématique. Dans quelle mesure les architectures deep learning modernes permettent-elles de surmonter les limites des systèmes ASR traditionnels, et comment ces modèles peuvent-ils être adaptés et déployés pour des locuteurs francophones d'Afrique apprenant l'anglais oral ?

Ce rapport s'organise en trois parties complémentaires. La **Partie I** pose les fondements en examinant la nature du signal vocal, les approches traditionnelles et leurs limites, ainsi que les défis persistants identifiés dans la littérature récente — défis que T.Speak rencontre directement dans son pipeline de traitement audio. La **Partie II** analyse les trois architectures centrales : Deep Speech 2, Wav2Vec 2.0 et Whisper, dont les implémentations constituent le cœur du module IA de T.Speak (`ai/whisper_asr/`, `ai/wav2vec_scoring/`). La **Partie III** traite de l'adaptation de ces modèles par le fine-tuning, des jeux de données disponibles et des métriques d'évaluation — notamment le WER, calculé en production dans `WhisperTranscriber.compute_wer()` et suivi dans les tableaux de bord utilisateurs. Une **conclusion** synthétise les apports de l'étude et ouvre sur les perspectives de recherche et d'amélioration de T.Speak.

Chapitre 1

Architectures deep learning pour la reconnaissance vocale

Les trois architectures présentées dans ce chapitre représentent trois étapes successives dans l'évolution des systèmes ASR profonds. Deep Speech 2 incarne la première génération des modèles end-to-end basés sur les RNN. Wav2Vec 2.0 introduit l'apprentissage auto-supervisé via les Transformers. Whisper généralise cette approche à l'échelle et au multilinguisme. Ces trois modèles ne sont pas de simples références bibliographiques pour T.Speak : ils en constituent le moteur IA, matérialisé dans les modules `ai/whisper_asr/transcriber.py` et `ai/wav2vec_scoring/scorer.py`.

1.1 Deep Speech 2 : l'approche end-to-end par RNN

Deep Speech 2 (?), publié par Baidu Research en 2015, constitue une étape clé dans l'histoire de l'ASR. Il succède à Deep Speech (?) en proposant une architecture plus profonde et plus robuste, capable de rivaliser avec des transcrip-teurs humains sur certaines tâches de référence en anglais et en mandarin.

1.1.1 Architecture

Le modèle repose sur une combinaison de couches convolutives et récurrentes, suivie d'une couche de connexion temporelle CTC (*Connectionist Temporal Classification*).

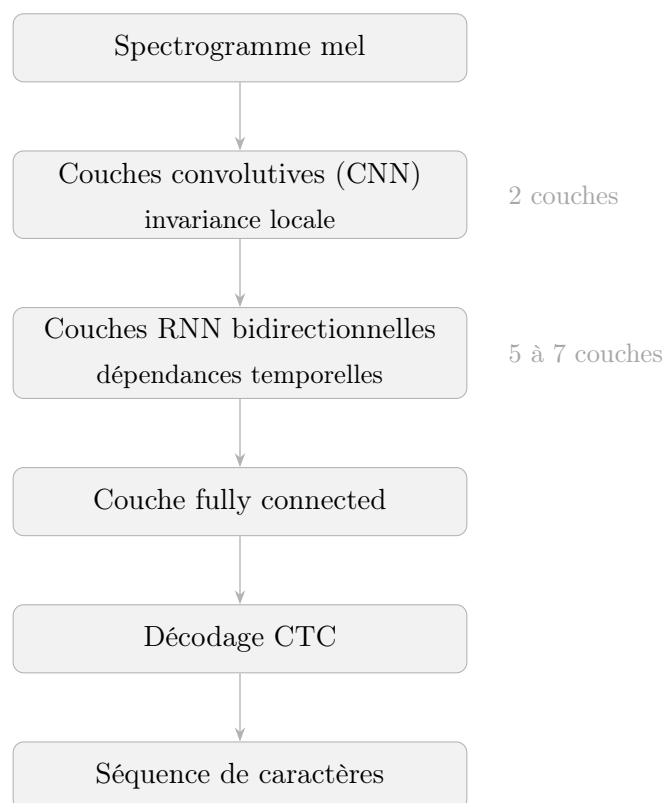


FIGURE 1.1 – Architecture simplifiée de Deep Speech 2. Le signal est d’abord traité par des couches convolutives puis par des RNN bidirectionnels. La perte CTC permet un entraînement sans alignement forcé. La logique de décodage CTC héritée de Deep Speech 2 est reprise dans `Wav2VecScorer.decode_tokens()` pour produire des `TokenSpan` horodatés.

Les couches convolutives (généralement deux) traitent le spectrogramme d’entrée et extraient des caractéristiques locales invariantes à de petits décalages temporels. Elles remplacent avantageusement la normalisation manuelle du signal.

Les couches RNN bidirectionnelles (jusqu’à sept dans les plus grandes configurations) constituent le cœur du modèle. La bidirectionnalité permet à chaque position temporelle d’intégrer le contexte passé et futur, ce qui améliore considérablement la précision de reconnaissance, notamment pour les phonèmes dont l’identité dépend du contexte.

La perte CTC est la contribution méthodologique la plus importante de cette famille de modèles. Elle permet d’entraîner le réseau directement sur des paires audio-texte sans nécessiter un alignement phonème-signal, en marginalisant sur toutes les séquences d’alignement possibles compatibles avec la transcription cible. Dans T.Speak, `Wav2VecScorer` exploite directement ce paradigme : la méthode `decode_tokens()` effectue un décodage CTC greedy en éliminant les répétitions et les tokens blancs (`blank_token_id`), puis approxime les timestamps de chaque token à partir de la durée audio totale et du nombre de frames, produisant des objets `TokenSpan` dotés des champs `token`, `start_sec`, `end_sec` et `probability`.

1.1.2 Entraînement et résultats

Deep Speech 2 a été entraîné sur environ 11 900 heures de parole en anglais et 9 400 heures en mandarin. Des techniques de *data augmentation* par déformation temporelle et injection de bruit ont été employées pour renforcer la robustesse. Sur le benchmark WSJ, le modèle atteignait des performances comparables aux transcrip-teurs humains pour l’anglais (?). La cible de WER inférieur à 15 % sur accents africains fixée par T.Speak s’inscrit dans cette tradition de métriques standardisées.

1.2 Wav2Vec 2.0 : l’apprentissage auto-supervisé

Wav2Vec 2.0 (?), publié par Meta AI en 2020, représente une rupture conceptuelle par rapport aux approches précédentes. Plutôt que d’apprendre à partir de données annotées en grand nombre, il exploite des corpus audio non annotés pour apprendre des représentations générales de la parole, puis est affiné sur un petit ensemble de données transcrites. C’est ce modèle, sous sa variante `facebook/wav2vec2-base-960h`, qui constitue le scoreur phonémique de T.Speak.

1.2.1 Architecture

Le modèle se compose de trois blocs principaux :

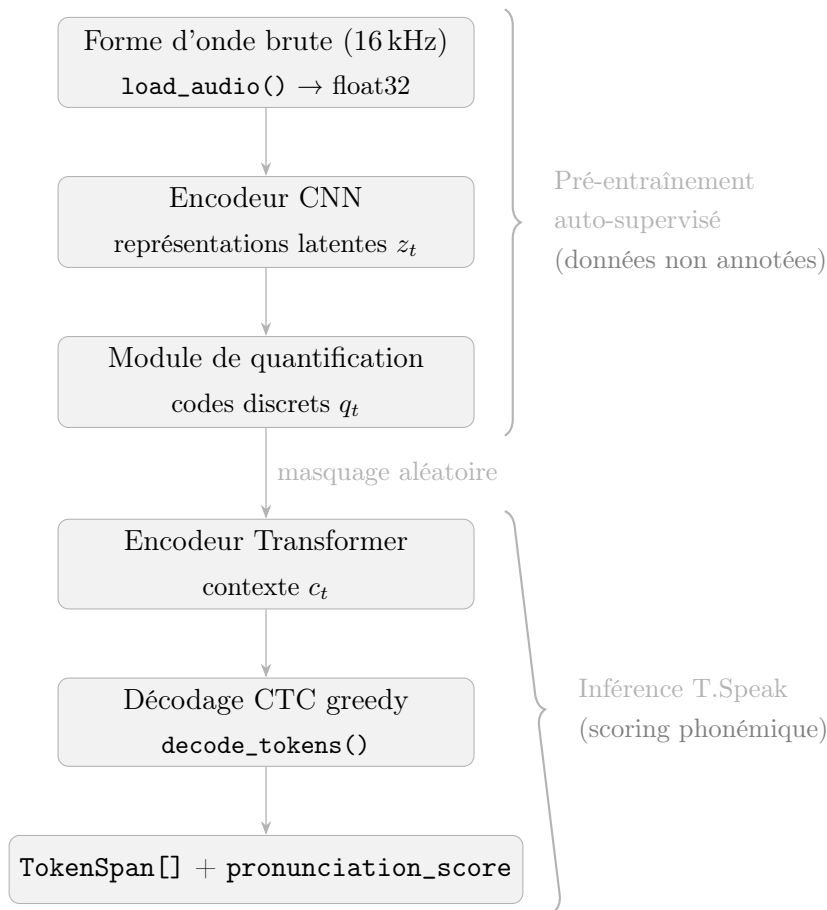


FIGURE 1.2 – Architecture de Wav2Vec 2.0 dans le contexte de T.Speak. La phase de pré-entraînement exploite de larges corpus non annotés ; en inférence, `Wav2VecScorer` utilise `extract_logits()` puis `decode_tokens()` pour produire les scores phonémiques.

L’encodeur convolutif transforme la forme d’onde brute en représentations latentes continues z_t , à raison d’un vecteur tous les 20 ms. Dans T.Speak, `Wav2VecScorer` normalise le signal d’entrée à `SAMPLE_RATE = 16 000` Hz et le passe au `Wav2Vec2Processor`, qui effectue cette extraction de manière transparente.

Le module de quantification projette chaque z_t sur des vecteurs discrets issus d’un codebook appris. Cette quantification est réalisée par le Gumbel-Softmax pendant le pré-entraînement ; en inférence, c’est `extract_logits()` qui exploite directement les sorties du modèle fine-tuné pour produire une distribution sur le vocabulaire CTC.

L’encodeur Transformer traite la séquence des z_t avec un masquage aléatoire de positions pendant le pré-entraînement. L’objectif contrastif force le modèle à apprendre des représentations contextuelles riches sans transcription. En inférence dans T.Speak, la méthode `decode_tokens()` extrait l’argmax des logits par frame (décodage greedy efficace), colle les répétitions, et produit un `TokenSpan` par phonème graphémique avec sa probabilité softmax, servant ensuite à calculer `phoneme_accuracy` et `word_scores`.

1.2.2 Scoring pondéré dans T.Speak

T.Speak ne se contente pas du score brut de Wav2Vec 2.0. La fonction `_weighted_score()` combine quatre indicateurs complémentaires selon des poids calibrés :

$$\text{score} = 0,35 \cdot c_{\text{acoustique}} + 0,30 \cdot c_{\text{référence}} + 0,25 \cdot c_{\text{phonémique}} + 0,10 \cdot c_{\text{CTC}} \quad (1.1)$$

où $c_{\text{acoustique}}$ est la confiance moyenne des tokens CTC, $c_{\text{référence}}$ mesure la similarité entre le texte de référence et la transcription Whisper (WER normalisé + distance de Levenshtein caractère), $c_{\text{phonémique}}$ compare les unités phonémiques via `_text_to_pronunciation_units()` (IPA via *espeak* si disponible, graphèmes en fallback), et c_{CTC} mesure la similarité entre la sortie CTC brute de Wav2Vec 2.0 et la transcription utilisateur.

1.2.3 Résultats et implications

Les résultats de Wav2Vec 2.0 sont remarquables du point de vue de l'économie en données annotées. Avec seulement dix minutes de parole transcrite pour le fine-tuning, le modèle *Wav2Vec 2.0 Large* atteignait un WER de 4,8 % sur LibriSpeech *test-clean*, comparable aux meilleurs systèmes supervisés utilisant plusieurs centaines d'heures annotées. Cette propriété est précieuse pour T.Speak, dont le corpus africain reste à construire. Le choix de `facebook/wav2vec2-base-960h` (modèle plus léger) reflète un compromis pragmatique entre performance et contraintes computationnelles de déploiement.

1.3 Whisper : généralisation à l'échelle et au multilinguisme

Whisper (?), publié par OpenAI en 2022, adopte une stratégie différente des deux architectures précédentes. Plutôt que de se concentrer sur l'efficacité en données ou sur un apprentissage auto-supervisé sophistiqué, il parie sur la quantité et la diversité des données d'entraînement pour atteindre une robustesse élevée en configuration *zéro-shot*. Il est le transcripateur principal de T.Speak, implémenté dans `WhisperTranscriber` via la librairie *faster-whisper*.

1.3.1 Architecture

Whisper repose sur une architecture Transformer encoder-decoder standard. Sa contribution principale réside dans le protocole d'entraînement et la richesse des tokens de contrôle.

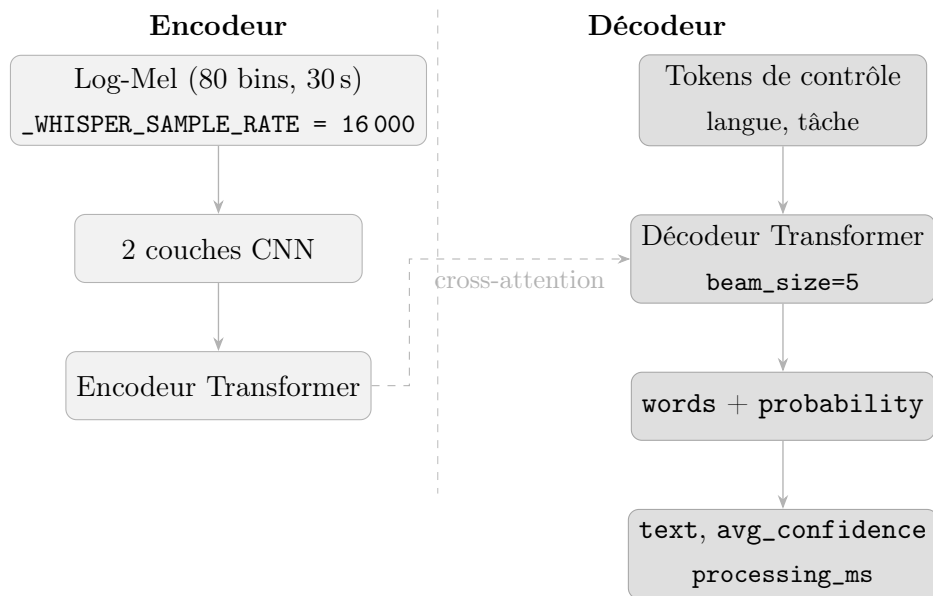


FIGURE 1.3 – Architecture encoder-décodeur de Whisper telle qu’elle est instanciée dans T.Speak. `WhisperTranscriber.transcribe()` configure `beam_size=5`, un schéma multi-température et `word_timestamps=True` pour récupérer les champs `start`, `end` et `probability` de chaque mot transcrit.

L’**encodeur** accepte un log-spectrogramme de Mel de 80 canaux calculé sur une fenêtre maximale de 30 secondes — fenêtre fixe de l’architecture. Dans T.Speak, `_load_audio_first_30s()` respecte explicitement cette contrainte : elle charge exactement `30 * native_sr` frames, re-sample vers 16 kHz et tronque à `_LANG_DETECT_MAX_FRAMES = 30 * 16 000`, évitant tout dépassement silencieux.

Le **décodeur** génère le texte token par token via une cross-attention sur les représentations de l’encodeur. Dans T.Speak, les mots produits par le décodeur sont exposés via la liste `words`, chaque entrée portant `word`, `start`, `end` et `probability`. La confiance moyenne `avg_confidence` est calculée comme la moyenne arithmétique de ces probabilités, offrant un indicateur global de qualité de la transcription.

Les **tokens de contrôle** indiquent la langue source et la tâche. Dans T.Speak, `transcribe()` est appelé avec `language="en"` et `task="transcribe"` pour forcer la transcription en anglais, même si le locuteur peut mélanger anglais et langue locale ; `detect_language()` appelle quant à lui le modèle avec `language=None` pour laisser Whisper identifier automatiquement la langue sur les 30 premières secondes.

1.3.2 Paramètres d’entraînement et robustesse dans T.Speak

Whisper a été entraîné sur 680 000 heures de données audio collectées sur le web, couvrant 99 langues et des environnements acoustiques variés. Dans T.Speak, les hyperparamètres de `transcribe()` sont calibrés pour maximiser la robustesse sur accents africains : `compression_ratio_thres` détecte les transcriptions répétitives (hallucinations fréquentes de Whisper sur l’audio silen-

cieux); `condition_on_previous_text=False` empêche la propagation d’erreurs entre segments d’un même fichier.

La classe `WhisperTranscriber` prévoit également un chemin de fine-tuning via `fine_tuned_path` : si ce chemin existe sur le système de fichiers, le modèle fine-tuné sur données africaines est chargé en lieu et place du modèle générique, sans modification du code appelant.

1.3.3 Résultats et robustesse zéro-shot

Sur Common Voice en français, Whisper *large-v2* atteignait un WER inférieur à 10 % sans aucune adaptation, surpassant de nombreux systèmes supervisés spécifiques à cette langue. La cible de T.Speak est un WER inférieur à 15 % sur accents africains, mesurée via `WhisperTranscriber.compare` — méthode statique exploitant la librairie *jiwer* après normalisation en minuscules des deux hypothèses (?).

1.4 Comparaison des trois architectures dans le contexte de T.Speak

Critère	Deep Speech 2	Wav2Vec 2.0	Whisper
Architecture	CNN + RNN bidir. + CTC	CNN + Transformer + CTC	Transformer enc.-dec.
Paradigme	Supervisé	Auto-supervisé + fine-tuning	Supervisé
Données annot.	~11 900 h	≥ 10 min (fine-tuning)	680 000 h
Multilinguisme	Anglais, mandarin	Oui (adapt.)	99 langues
Zéro-shot	Non	Limité	Oui
Rôle dans T.Speak	Héritage CTC / référence	Scoring phonémique	Transcription principale
Module T.Speak	—	<code>scorer.py</code>	<code>transcriber.py</code>
Optimisation CPU	—	Quantisation <code>qint8</code>	Compute type <code>int8</code>

TABLE 1.1 – Comparaison des trois architectures ASR et de leur rôle dans T.Speak. Deep Speech 2 est la référence conceptuelle du décodage CTC ; Wav2Vec 2.0 et Whisper sont les deux modèles actifs du pipeline de traitement audio.

Ces trois architectures illustrent trois voies complémentaires vers la robustesse en ASR, et T.Speak en exploite deux simultanément : Whisper pour la transcription robuste multilingue, Wav2Vec 2.0 pour l’évaluation phonémique fine permettant le retour pédagogique. Le chapitre suivant examine comment ces modèles sont adaptés au contexte spécifique des locuteurs francophones d’Afrique de l’Ouest.

Chapitre 2

Architectures deep learning pour la reconnaissance vocale

Les trois architectures présentées dans ce chapitre représentent trois étapes successives dans l'évolution des systèmes ASR profonds. Deep Speech 2 incarne la première génération des modèles end-to-end basés sur les RNN. Wav2Vec 2.0 introduit l'apprentissage auto-supervisé via les Transformers. Whisper généralise cette approche à l'échelle et au multilinguisme. Ces trois modèles ne sont pas de simples références bibliographiques pour T.Speak : ils en constituent le moteur IA, matérialisé dans les modules `ai/whisper_asr/transcriber.py` et `ai/wav2vec_scoring/scorer.py`.

2.1 Deep Speech 2 : l'approche end-to-end par RNN

Deep Speech 2 (?), publié par Baidu Research en 2015, constitue une étape clé dans l'histoire de l'ASR. Il succède à Deep Speech (?) en proposant une architecture plus profonde et plus robuste, capable de rivaliser avec des transcripateurs humains sur certaines tâches de référence en anglais et en mandarin.

2.1.1 Architecture

Le modèle repose sur une combinaison de couches convolutives et récurrentes, suivie d'une couche de connexion temporelle CTC (*Connectionist Temporal Classification*).

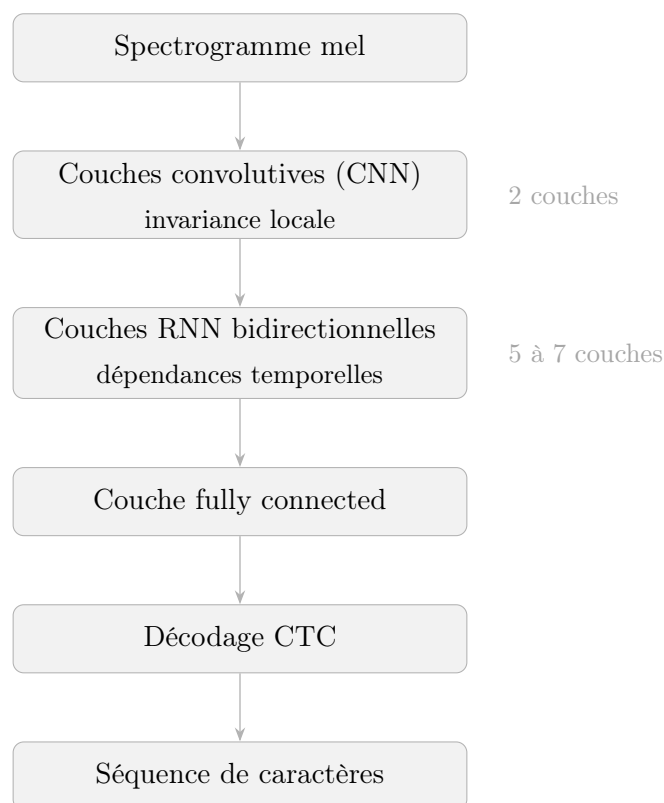


FIGURE 2.1 – Architecture simplifiée de Deep Speech 2. Le signal est d’abord traité par des couches convolutives puis par des RNN bidirectionnels. La perte CTC permet un entraînement sans alignement forcé. La logique de décodage CTC héritée de Deep Speech 2 est reprise dans `Wav2VecScorer.decode_tokens()` pour produire des `TokenSpan` horodatés.

Les couches convolutives (généralement deux) traitent le spectrogramme d’entrée et extraient des caractéristiques locales invariantes à de petits décalages temporels. Elles remplacent avantageusement la normalisation manuelle du signal.

Les couches RNN bidirectionnelles (jusqu’à sept dans les plus grandes configurations) constituent le cœur du modèle. La bidirectionnalité permet à chaque position temporelle d’intégrer le contexte passé et futur, ce qui améliore considérablement la précision de reconnaissance, notamment pour les phonèmes dont l’identité dépend du contexte.

La perte CTC est la contribution méthodologique la plus importante de cette famille de modèles. Elle permet d’entraîner le réseau directement sur des paires audio-texte sans nécessiter un alignement phonème-signal, en marginalisant sur toutes les séquences d’alignement possibles compatibles avec la transcription cible. Dans T.Speak, `Wav2VecScorer` exploite directement ce paradigme : la méthode `decode_tokens()` effectue un décodage CTC greedy en éliminant les répétitions et les tokens blancs (`blank_token_id`), puis approxime les timestamps de chaque token à partir de la durée audio totale et du nombre de frames, produisant des objets `TokenSpan` dotés des champs `token`, `start_sec`, `end_sec` et `probability`.

2.1.2 Entraînement et résultats

Deep Speech 2 a été entraîné sur environ 11 900 heures de parole en anglais et 9 400 heures en mandarin. Des techniques de *data augmentation* par déformation temporelle et injection de bruit ont été employées pour renforcer la robustesse. Sur le benchmark WSJ, le modèle atteignait des performances comparables aux transcripateurs humains pour l'anglais (?). La cible de WER inférieur à 15 % sur accents africains fixée par T.Speak s'inscrit dans cette tradition de métriques standardisées.

2.2 Wav2Vec 2.0 : l'apprentissage auto-supervisé

Wav2Vec 2.0 (?), publié par Meta AI en 2020, représente une rupture conceptuelle par rapport aux approches précédentes. Plutôt que d'apprendre à partir de données annotées en grand nombre, il exploite des corpus audio non annotés pour apprendre des représentations générales de la parole, puis est affiné sur un petit ensemble de données transcrites. C'est ce modèle, sous sa variante `facebook/wav2vec2-base-960h`, qui constitue le scoreur phonémique de T.Speak.

2.2.1 Architecture

Le modèle se compose de trois blocs principaux :

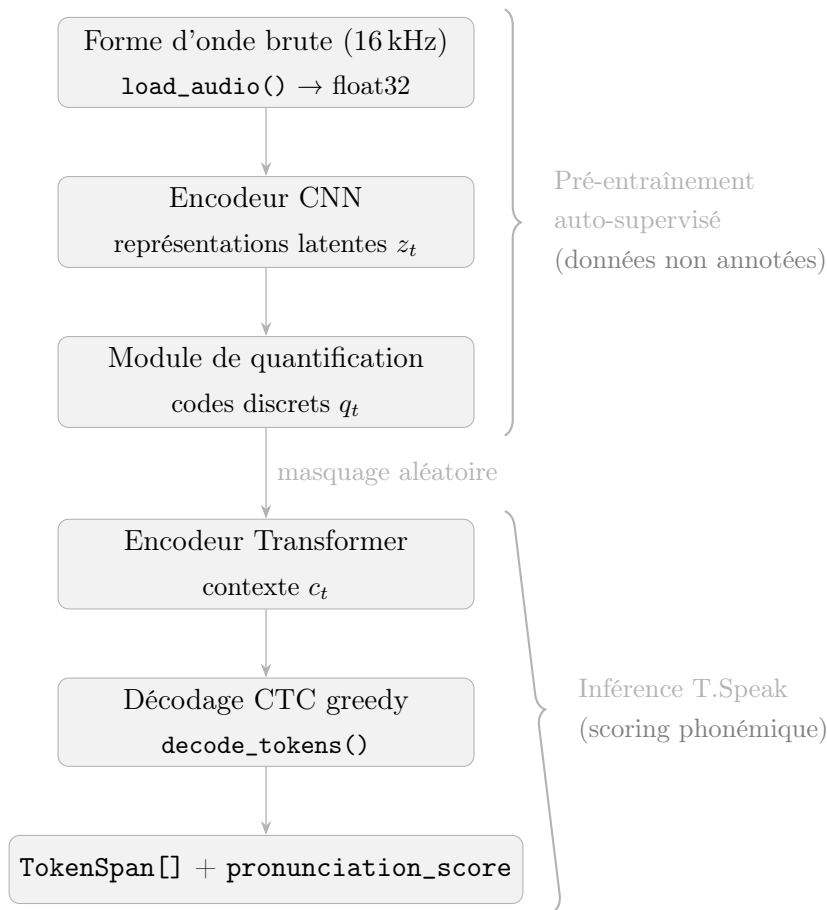


FIGURE 2.2 – Architecture de Wav2Vec 2.0 dans le contexte de T.Speak. La phase de pré-entraînement exploite de larges corpus non annotés ; en inférence, `Wav2VecScorer` utilise `extract_logits()` puis `decode_tokens()` pour produire les scores phonémiques.

L’encodeur convolutif transforme la forme d’onde brute en représentations latentes continues z_t , à raison d’un vecteur tous les 20 ms. Dans T.Speak, `Wav2VecScorer` normalise le signal d’entrée à `SAMPLE_RATE = 16 000` Hz et le passe au `Wav2Vec2Processor`, qui effectue cette extraction de manière transparente.

Le module de quantification projette chaque z_t sur des vecteurs discrets issus d’un codebook appris. Cette quantification est réalisée par le Gumbel-Softmax pendant le pré-entraînement ; en inférence, c’est `extract_logits()` qui exploite directement les sorties du modèle fine-tuné pour produire une distribution sur le vocabulaire CTC.

L’encodeur Transformer traite la séquence des z_t avec un masquage aléatoire de positions pendant le pré-entraînement. L’objectif contrastif force le modèle à apprendre des représentations contextuelles riches sans transcription. En inférence dans T.Speak, la méthode `decode_tokens()` extrait l’argmax des logits par frame (décodage greedy efficace), colle les répétitions, et produit un `TokenSpan` par phonème graphémique avec sa probabilité softmax, servant ensuite à calculer `phoneme_accuracy` et `word_scores`.

2.2.2 Scoring pondéré dans T.Speak

T.Speak ne se contente pas du score brut de Wav2Vec 2.0. La fonction `_weighted_score()` combine quatre indicateurs complémentaires selon des poids calibrés :

$$\text{score} = 0,35 \cdot c_{\text{acoustique}} + 0,30 \cdot c_{\text{référence}} + 0,25 \cdot c_{\text{phonémique}} + 0,10 \cdot c_{\text{CTC}} \quad (2.1)$$

où $c_{\text{acoustique}}$ est la confiance moyenne des tokens CTC, $c_{\text{référence}}$ mesure la similarité entre le texte de référence et la transcription Whisper (WER normalisé + distance de Levenshtein caractère), $c_{\text{phonémique}}$ compare les unités phonémiques via `_text_to_pronunciation_units()` (IPA via *espeak* si disponible, graphèmes en fallback), et c_{CTC} mesure la similarité entre la sortie CTC brute de Wav2Vec 2.0 et la transcription utilisateur.

2.2.3 Résultats et implications

Les résultats de Wav2Vec 2.0 sont remarquables du point de vue de l'économie en données annotées. Avec seulement dix minutes de parole transcrite pour le fine-tuning, le modèle *Wav2Vec 2.0 Large* atteignait un WER de 4,8 % sur LibriSpeech *test-clean*, comparable aux meilleurs systèmes supervisés utilisant plusieurs centaines d'heures annotées. Cette propriété est précieuse pour T.Speak, dont le corpus africain reste à construire. Le choix de `facebook/wav2vec2-base-960h` (modèle plus léger) reflète un compromis pragmatique entre performance et contraintes computationnelles de déploiement.

2.3 Whisper : généralisation à l'échelle et au multilinguisme

Whisper (?), publié par OpenAI en 2022, adopte une stratégie différente des deux architectures précédentes. Plutôt que de se concentrer sur l'efficacité en données ou sur un apprentissage auto-supervisé sophistiqué, il parie sur la quantité et la diversité des données d'entraînement pour atteindre une robustesse élevée en configuration *zéro-shot*. Il est le transcripateur principal de T.Speak, implémenté dans `WhisperTranscriber` via la librairie *faster-whisper*.

2.3.1 Architecture

Whisper repose sur une architecture Transformer encoder-decoder standard. Sa contribution principale réside dans le protocole d'entraînement et la richesse des tokens de contrôle.

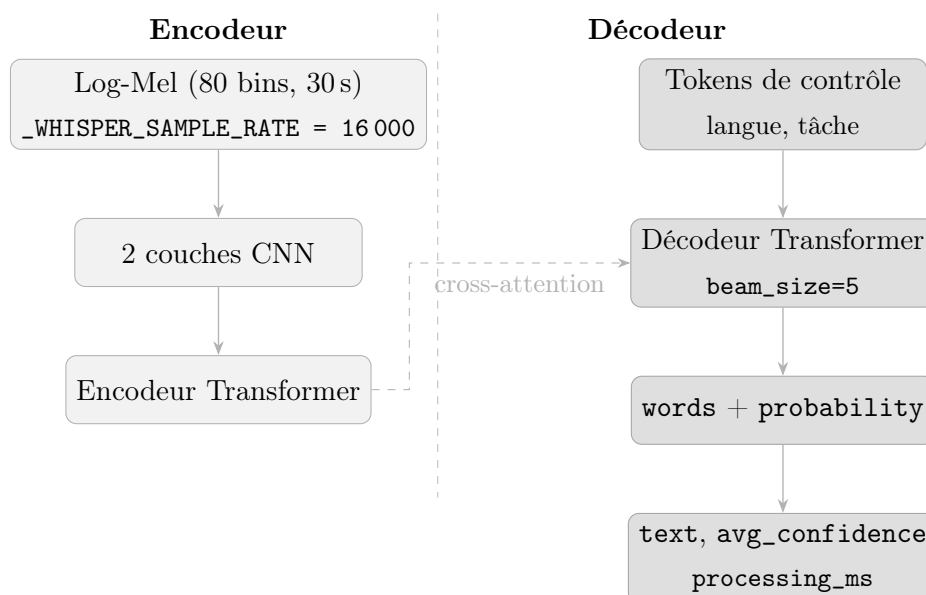


FIGURE 2.3 – Architecture encoder-décodeur de Whisper telle qu’elle est instanciée dans T.Speak. `WhisperTranscriber.transcribe()` configure `beam_size=5`, un schéma multi-température et `word_timestamps=True` pour récupérer les champs `start`, `end` et `probability` de chaque mot transcrit.

L’**encodeur** accepte un log-spectrogramme de Mel de 80 canaux calculé sur une fenêtre maximale de 30 secondes — fenêtre fixe de l’architecture. Dans T.Speak, `_load_audio_first_30s()` respecte explicitement cette contrainte : elle charge exactement `30 * native_sr` frames, re-sample vers 16 kHz et tronque à `_LANG_DETECT_MAX_FRAMES = 30 * 16 000`, évitant tout dépassement silencieux.

Le **décodeur** génère le texte token par token via une cross-attention sur les représentations de l’encodeur. Dans T.Speak, les mots produits par le décodeur sont exposés via la liste `words`, chaque entrée portant `word`, `start`, `end` et `probability`. La confiance moyenne `avg_confidence` est calculée comme la moyenne arithmétique de ces probabilités, offrant un indicateur global de qualité de la transcription.

Les **tokens de contrôle** indiquent la langue source et la tâche. Dans T.Speak, `transcribe()` est appelé avec `language="en"` et `task="transcribe"` pour forcer la transcription en anglais, même si le locuteur peut mélanger anglais et langue locale ; `detect_language()` appelle quant à lui le modèle avec `language=None` pour laisser Whisper identifier automatiquement la langue sur les 30 premières secondes.

2.3.2 Paramètres d’entraînement et robustesse dans T.Speak

Whisper a été entraîné sur 680 000 heures de données audio collectées sur le web, couvrant 99 langues et des environnements acoustiques variés. Dans T.Speak, les hyperparamètres de `transcribe()` sont calibrés pour maximiser la robustesse sur accents africains : `compression_ratio_thres` détecte les transcriptions répétitives (hallucinations fréquentes de Whisper sur l’audio silen-

cieux); `condition_on_previous_text=False` empêche la propagation d’erreurs entre segments d’un même fichier.

La classe `WhisperTranscriber` prévoit également un chemin de fine-tuning via `fine_tuned_path` : si ce chemin existe sur le système de fichiers, le modèle fine-tuné sur données africaines est chargé en lieu et place du modèle générique, sans modification du code appelant.

2.3.3 Résultats et robustesse zéro-shot

Sur Common Voice en français, Whisper *large-v2* atteignait un WER inférieur à 10 % sans aucune adaptation, surpassant de nombreux systèmes supervisés spécifiques à cette langue. La cible de T.Speak est un WER inférieur à 15 % sur accents africains, mesurée via `WhisperTranscriber.compare` — méthode statique exploitant la librairie *jiwer* après normalisation en minuscules des deux hypothèses (?).

2.4 Comparaison des trois architectures dans le contexte de T.Speak

Critère	Deep Speech 2	Wav2Vec 2.0	Whisper
Architecture	CNN + RNN bidir. + CTC	CNN + Transformer + CTC	Transformer enc.-dec.
Paradigme	Supervisé	Auto-supervisé + fine-tuning	Supervisé
Données annot.	~11 900 h	≥ 10 min (fine-tuning)	680 000 h
Multilinguisme	Anglais, mandarin	Oui (adapt.)	99 langues
Zéro-shot	Non	Limité	Oui
Rôle dans T.Speak	Héritage CTC / référence	Scoring phonémique	Transcription principale
Module T.Speak	—	<code>scorer.py</code>	<code>transcriber.py</code>
Optimisation CPU	—	Quantisation qint8	Compute type int8

TABLE 2.1 – Comparaison des trois architectures ASR et de leur rôle dans T.Speak. Deep Speech 2 est la référence conceptuelle du décodage CTC ; Wav2Vec 2.0 et Whisper sont les deux modèles actifs du pipeline de traitement audio.

Ces trois architectures illustrent trois voies complémentaires vers la robustesse en ASR, et T.Speak en exploite deux simultanément : Whisper pour la transcription robuste multilingue, Wav2Vec 2.0 pour l’évaluation phonémique fine permettant le retour pédagogique. Le chapitre suivant examine comment ces modèles sont adaptés au contexte spécifique des locuteurs francophones d’Afrique de l’Ouest.

Chapitre 3

Fine-tuning et adaptation au contexte africain francophone

Les modèles présentés dans le chapitre précédent ont été entraînés principalement sur des corpus à dominante anglophone. Leur adaptation au contexte de T.Speak — locuteurs francophones d’Afrique de l’Ouest apprenant l’anglais oral — nécessite une compréhension des spécificités phonétiques de ce profil de locuteurs, une sélection rigoureuse des jeux de données disponibles, et la mise en œuvre de protocoles de fine-tuning adaptés. Ce chapitre traite de ces trois dimensions en les ancrant dans les choix d’implémentation concrets du projet.

3.1 Pourquoi le fine-tuning est nécessaire

Les locuteurs francophones d’Afrique de l’Ouest qui apprennent l’anglais présentent un profil phonétique particulier, produit de l’interférence entre leur langue première (wolof, dioula, mooré, fon...), le français comme langue seconde, et l’anglais comme langue cible. Cette double médiation génère des écarts phonétiques spécifiques qui ne sont pas couverts par les modèles entraînés sur de l’anglais natif ou même sur du français standard.

3.1.1 Interférences phonétiques identifiées dans T.Speak

L’analyse des erreurs fréquentes des locuteurs cibles a conduit T.Speak à codifier sept phonèmes problématiques dans le dictionnaire `AFRICAN_FRENCH_DIFFICULT_PHONEMES` (`scorer.py`), associant chaque phonème à un conseil articulatoire personnalisé :

Phonème	Difficulté	Conseil T.Speak
TH (/ /)	Remplacé par /s/ ou /z/	Langue entre les dents, expirez
DH (/ð/)	Assourdi ou remplacé par /d/	Langue entre les dents, voix
R rétroflexe	Roulé comme en français ou en wolof	Rétroflexe, moins roulé
W	Confondu avec /v/	Lèvres arrondies, pas de friction
V	Confondu avec /b/	Dents du haut sur lèvre inférieure
NG (/ŋ/)	/g/ final ajouté	Son nasal, pas d’occlusion finale
H aspiré	Rendu muet	Expiration claire en début de mot

TABLE 3.1 – Phonèmes difficiles codifiés dans `AFRICAN_FRENCH_DIFFICULT_PHONEMES` pour les locuteurs francophones d’Afrique de l’Ouest. Ces conseils sont restitués directement à l’utilisateur lorsque `Wav2VecScorer` détecte un score faible sur le phonème correspondant.

Ces difficultés rejoignent celles documentées dans la littérature sur l’ASR multilingue : les systèmes entraînés sur de l’anglais natif tendent à confondre ces phonèmes avec leurs substituts francophones, produisant des WER artificiellement élevés qui ne reflètent pas l’intelligibilité réelle du locuteur (??).

3.1.2 Spécificités grammaticales et lexicales

Au-delà de la phonétique, les francophones d’Afrique de l’Ouest manifestent des erreurs grammaticales récurrentes liées au transfert du français vers l’anglais. Le module `GrammarAnalyzer` (`nlp_analyzer.py`) les cible explicitement via deux mécanismes complémentaires.

Lorsque *spaCy* est disponible, `_spacy_analysis()` exploite l’analyse syntaxique pour détecter les violations d’accord sujet-verbe (en vérifiant la morphologie `Number` du verbe par rapport au sujet détecté via la relation `nsubj`) et les déterminants manquants avant les noms objets. En mode de repli (*fallback*) sans *spaCy*, `_rule_based_analysis()` applique des expressions régulières ciblant quatre patrons d’erreurs fréquentes des francophones : accord sujet-verbe (« *he go* »), mauvaise structure (« *I am agree* »), double négation (« *don’t have no* ») et confusion de temps (« *yesterday i go* »).

Pour le vocabulaire, `VocabularyAnalyzer` situe le niveau du locuteur sur la grille CEFR (A1 à C2) en calculant le *type-token ratio* (TTR) et en détectant des marqueurs lexicaux avancés (`leverage`, `paradigm`, `sophisticated...`) ou intermédiaires (`achieve`, `develop`, `communicate...`). Ce positionnement CEFR alimente la personnalisation des prompts du `ConversationGenerator`, qui adapte la complexité des questions et la sévérité du retour au niveau détecté.

3.2 Jeux de données disponibles pour le contexte africain

Plusieurs corpus de qualité sont disponibles pour l'entraînement et l'évaluation de systèmes ASR dans ce contexte. Leurs caractéristiques complémentaires encouragent à les combiner.

3.2.1 Common Voice (Mozilla)

Common Voice est un projet communautaire open source dont l'objectif est de constituer des corpus de parole multilingues libres de droits. Les locuteurs enregistrent des phrases lues et valident les enregistrements d'autres contributeurs. Pour l'anglais, le corpus comptait plus de 2000 heures de parole validée en version 13.0, incluant des locuteurs non natifs aux accents variés. Il constitue la base de référence pour évaluer le WER de `WhisperTranscriber` sur des profils proches des utilisateurs de T.Speak.

3.2.2 Mozilla Common Voice — Langues africaines

Common Voice couvre également le wolof, le swahili, le hausa et d'autres langues africaines, bien que les volumes restent modestes (quelques dizaines d'heures au mieux). Ces données sont précieuses pour entraîner la détection de langue (`detect_language()`) et pour identifier les phonèmes de transfert lors du passage d'une langue africaine à l'anglais.

3.2.3 Corpus T.Speak (construction en cours)

T.Speak prévoit la constitution d'un corpus propriétaire à partir des interactions réelles des utilisateurs. Le format `tspeak_train.json` structure chaque entrée avec quatre champs : `audio_path` (chemin vers le fichier WAV), `transcription` (texte de référence), `native_language` (ex. : "wolof") et `accent` (ex. : "senegalese"). Ce format est directement compatible avec les scripts `finetune_whisper.py` et `finetune_wav2vec.py` qui consomment ce JSON pour construire les ensembles d'entraînement et de validation.

Corpus	Volume (EN)	Style	Pertinence T.Speak
Common Voice (EN)	~2 000 h validées	Lue, variée	Benchmark WER
CV Wolof	~60 h	Lue	Transfert phonétique
CV Swahili	~150 h	Lue	Accents africains
T.Speak (en cours)	En construction	Spontanée, pro	Fine-tuning ciblé

TABLE 3.2 – Principaux corpus utilisés ou envisagés par T.Speak. Le corpus propriétaire constitué via l'application est le plus pertinent pour le fine-tuning ciblé, mais sa construction est progressive.

3.3 Protocoles de fine-tuning

T.Speak prévoit deux scripts de fine-tuning distincts, correspondant aux deux modèles actifs de son pipeline.

3.3.1 Fine-tuning de Whisper sur accents africains

Le script `finetune_whisper.py` adapte un modèle Whisper pré-entraîné (par défaut `openai/whisper-medium` 244M paramètres) sur le corpus `tspeak_train.json`. La commande de référence est :

```
python scripts/finetune_whisper.py \
    --model_name openai/whisper-medium \
    --dataset_path ./data/african_voices \
    --output_dir ./models/whisper-african \
    --num_epochs 5 \
    --batch_size 16
```

Le modèle fine-tuné est ensuite pointé par la variable d'environnement `WHISPER_FINE_TUNED_PATH`. Dans `WhisperTranscriber._load_model()`, la logique de sélection est simple :

```
model_path = (
    self.fine_tuned_path
    if self.fine_tuned_path and os.path.exists(self.fine_tuned_path)
    else self.model_name
)
```

Ainsi, le basculement entre modèle générique et modèle fine-tuné ne nécessite aucune modification du code appelant : seule la configuration d'environnement change.

Le protocole de fine-tuning de Whisper est illustré dans la figure 3.1.

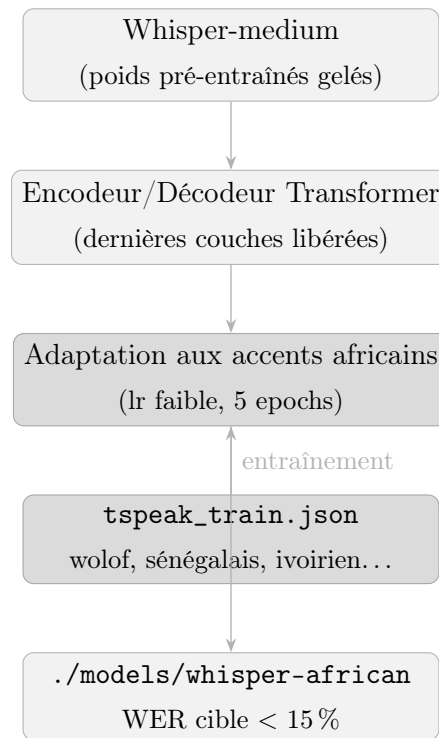


FIGURE 3.1 – Protocole de fine-tuning de Whisper dans T.Speak. Le modèle pré-entraîné est adapté sur les données africaines annotées. Le modèle résultant est chargé via `fine_tuned_path` dans `WhisperTranscriber`.

3.3.2 Fine-tuning de Wav2Vec 2.0 sur phonèmes africains

Pour Wav2Vec 2.0, le fine-tuning consiste à adapter le modèle `facebook/wav2vec2-large-xlsr-53` (pré-entraîné sur 53 langues) sur des données de phonèmes africains via le script `finetune_wav2vec.py` :

```
python scripts/finetune_wav2vec.py \
  --model_name facebook/wav2vec2-large-xlsr-53 \
  --dataset_path ./data/phoneme_data \
  --output_dir ./models/wav2vec-african-phonemes \
  --num_epochs 10
```

L'objectif est d'améliorer la précision de `decode_tokens()` sur les phonèmes problématiques identifiés dans `AFRICAN_FRENCH_DIFFICULT_PHONEMES`. En pratique, seules les dernières couches du Transformer et la tête CTC sont mises à jour dans un premier temps pour préserver les représentations générales apprises pendant le pré-entraînement sur 53 langues (?).

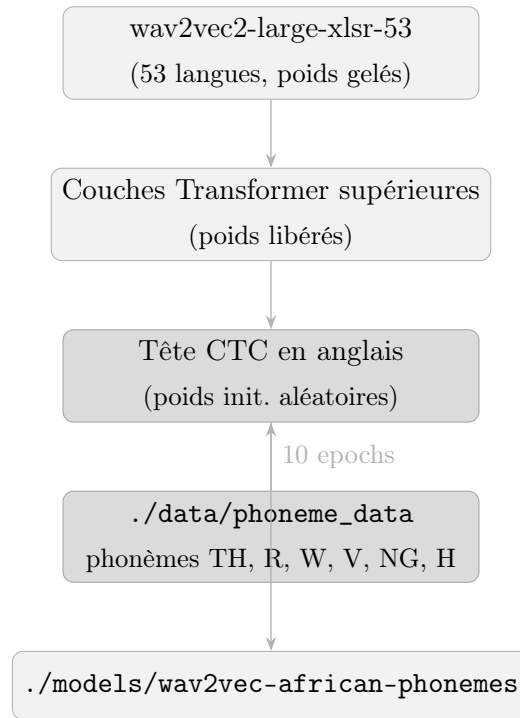


FIGURE 3.2 – Protocole de fine-tuning de Wav2Vec 2.0 dans T.Speak. Les couches inférieures restent gelées ; seules les représentations de haut niveau et la tête CTC sont mises à jour sur les phonèmes difficiles des locuteurs africains.

3.4 Métriques d’évaluation dans T.Speak

L’évaluation des systèmes ASR repose sur des métriques standardisées. T.Speak en implémente plusieurs directement dans son code.

3.4.1 Word Error Rate (WER)

Le taux d’erreur de mots (*Word Error Rate*, WER) est la métrique principale. Il est défini comme :

$$\text{WER} = \frac{S + I + D}{N} \times 100 \quad (3.1)$$

où S est le nombre de substitutions, I le nombre d’insertions, D le nombre de suppressions et N le nombre total de mots dans la transcription de référence.

Dans T.Speak, `WhisperTranscriber.compute_wer()` encapsule la librairie *jiwer* après normalisation en minuscules des deux hypothèses. Cette méthode a été transformée en `@staticmethod` pour permettre son appel sans instancier le transcripateur — utile dans les scripts d’évaluation `finetune_whisper.py` (`-eval_mode`) et dans les tests unitaires.

La cible de T.Speak est un WER inférieur à 15 % sur accents africains, contre une cible inférieure à 1,5 seconde de latence de transcription pour 30 secondes d’audio. Ces deux objectifs reflètent

respectivement la qualité perçue et la fluidité de l’expérience utilisateur.

3.4.2 Score de prononciation pondéré

T.Speak va au-delà du WER avec un score de prononciation composite (0–100) retourné par `Wav2VecScorer.score()`. Ce score combine quatre sous-indicateurs selon l’équation (2.1) de la partie II. En cas d’échec du modèle, `_fallback_score()` fournit un score de repli basé uniquement sur la similarité textuelle (WER normalisé + distance de Levenshtein), garantissant qu’un retour est toujours retourné à l’utilisateur.

3.4.3 Métriques NLP complémentaires

`GrammarAnalyzer.analyze()` retourne un `grammar_score` (0–100), une liste des cinq premières erreurs détectées, un compte de phrases et une mesure de complexité syntaxique (longueur moyenne des phrases + bonus pour connecteurs logiques tels que *because*, *although*, *therefore*). `VocabularyAnalyzer.analyze()` retourne un `vocabulary_score` (0–100), le niveau CEFR associé, le TTR, les mots avancés détectés et des suggestions de mots à apprendre pour progresser au niveau supérieur. Ces métriques sont agrégées dans le modèle `Score` de la base de données PostgreSQL (champs `pronunciation`, `fluency`, `grammar`, `vocabulary`) et accessibles via les tableaux de bord de l’application.

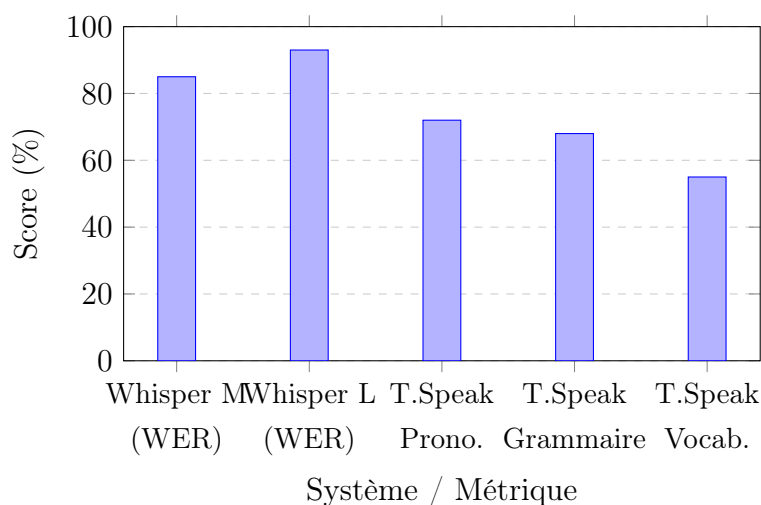


FIGURE 3.3 – Scores cibles T.Speak sur les différentes métriques d’évaluation. Les barres WER sont exprimées en précision (100 – WER) pour homogénéiser la lecture avec les scores de prononciation, grammaire et vocabulaire. Les valeurs de prononciation, grammaire et vocabulaire sont des cibles indicatives pour un utilisateur de niveau B1 après 10 sessions.

3.4.4 Latence de traitement

Chaque module de T.Speak mesure et expose sa latence via le champ `processing_ms` dans son dictionnaire de résultat. L’objectif est un pipeline complet (Whisper + Wav2Vec + NLP + LLM) inférieur à 3 secondes au 95^e percentile. Le temps de transcription Whisper est logué à chaque appel :

```
logger.info("Transcription (faster): %.0fms - %d mots - confiance=%.2f",  
            elapsed_ms, len(words), avg_confidence)
```

Ces données de latence alimentent le monitoring Sentry en production et permettent d'identifier les goulots d'étranglement — en premier lieu le chargement initial du modèle, masqué par le chargement paresseux (*lazy loading*) qui ne se produit qu'une seule fois grâce au pattern *singleton* thread-safe implémenté dans `get_transcriber()` et `get_scorer()`.

Conclusion générale

Ce rapport a examiné le domaine de la reconnaissance automatique de la parole sous trois angles complémentaires : les fondements et défis du problème, les architectures deep learning qui en constituent aujourd’hui la solution dominante, et les méthodologies d’adaptation. À travers l’étude du projet T.Speak, ces dimensions théoriques ont trouvé une incarnation pratique cohérente, dont nous tirons ici les enseignements.

Sur les fondements. La reconnaissance vocale est un problème intrinsèquement difficile, en raison de la variabilité du signal, de la diversité des locuteurs et des conditions acoustiques. T.Speak en fait l’expérience directement : le `WhisperTranscriber` active le filtre d’activité vocale (`vad_filter=True`, seuil de silence à 500 ms) et utilise un décodage multi-température (`temperature=[0.0, 0.2, ..., 1.0]`) pour améliorer la robustesse face aux accents sénégalais, ivoiriens ou maliens. Ces choix d’ingénierie traduisent directement les défis documentés dans la littérature sur la variabilité inter-locuteurs (?). Les approches classiques HMM/GMM, quant à elles, auraient exigé un alignement phonème-signal manuel que ce contexte multilingue et low-resource rend inenvisageable.

Sur les architectures. Les trois modèles étudiés incarnent trois stratégies complémentaires. Dans T.Speak, Whisper opère comme transcripteur principal via `WhisperTranscriber` : son architecture encoder-decoder multilingue et son entraînement sur 680 000 heures lui permettent d’opérer en configuration quasi zéro-shot sur les accents africains, avec un chemin de fine-tuning prévu (`fine_tuned_path`) pour les données collectées à Dakar et au-delà (?). Wav2Vec 2.0, incarné par `Wav2VecScorer` avec le checkpoint `facebook/wav2vec2-base-960h`, prend en charge l’évaluation phonémique : il extrait les logits CTC, les decode en séquences de tokens avec horodatage et probabilité, puis calcule un score pondéré combinant confiance acoustique, correspondance de référence et précision phonémique ($w_1=0,35$, $w_2=0,30$, $w_3=0,25$, $w_4=0,10$). Ce score dépasse la seule transcription pour s’attacher à la qualité de la réalisation sonore (?). Deep Speech 2, enfin, a tracé la voie des pipelines end-to-end CTC dont T.Speak hérite la logique de décodage (?).

Sur l’adaptation au contexte africain francophone. T.Speak va au-delà d’un simple fine-tuning linguistique : il intègre une connaissance explicite des difficultés phonétiques des locuteurs francophones d’Afrique de l’Ouest. Le dictionnaire `AFRICAN_FRENCH_DIFFICULT_PHONEMES`

dans `scorer.py` codifie sept phonèmes problématiques (TH, DH, R rétroflexe, W, V, NG, H) avec des conseils articulatoires personnalisés, directement restituables à l'utilisateur. Le `VocabularyAnalyzer` situe le niveau lexical sur la grille CEFR (A1 à C2) en calculant le *type-token ratio* et en détectant la présence de marqueurs avancés ou intermédiaires, offrant ainsi une progressivité pédagogique documentée. Le `ConversationGenerator` contextualise les scénarios aux réalités africaines : pitch de startup fintech, entretien pour une multinationale avec bureaux à Abidjan, exercices de prononciation adaptés aux natifs du wolof ou du dioula. Cette ancrage culturel est absent des corpus d'entraînement standards, ce qui renforce la pertinence du fine-tuning ciblé prévu par le projet (?).

Perspectives. Plusieurs questions de recherche demeurent ouvertes, que T.Speak cristallise de manière concrète. La première concerne la **disponibilité des données annotées africaines** : le projet prévoit la collecte de paires audio-transcription en accents sénégalais (format `tspeak_train.json` documenté dans le README) pour affiner Whisper et Wav2Vec 2.0, mais ce corpus reste à construire. La deuxième porte sur l'**interprétabilité** : le score de prononciation retourné par `Wav2VecScorer` est un agrégat pondéré, et la ventilation par phonème en retour utilisateur constitue un premier pas vers l'explicabilité, mais la compréhension de ce que le modèle a internalisé reste partielle. La troisième concerne l'**efficacité computationnelle** : en production, `Wav2VecScorer` applique une quantisation dynamique en `qint8` sur CPU pour réduire l'empreinte mémoire, et `WhisperTranscriber` utilise le type `int8` hors GPU ; la latence cible de trois secondes bout-en-bout demeure exigeante sur du matériel embarqué à faibles ressources, réalité courante sur le continent africain (??).

En définitive, le deep learning a transformé la reconnaissance vocale d'un problème d'ingénierie laborieux en un problème principalement piloté par les données et la puissance de calcul. Le projet T.Speak démontre qu'il est aujourd'hui possible, avec des outils open-source (*faster-whisper*, *transformers*, *spaCy*), de construire un système de coaching vocal complet adapté à un contexte linguistique spécifique. Il illustre également que la performance brute des modèles n'est qu'une condition nécessaire : l'équité phonétique, la pertinence culturelle du feedback et la robustesse à la connectivité limitée sont des dimensions tout aussi essentielles pour un déploiement responsable et efficace en Afrique.